

Learning Metamaterial Eigenmodes with Wavelet-Encoded Fourier Neural Operators

Han Zhang¹, Alexander Ogren¹, Cynthia Rudin¹, Johann Guilleminot¹, L. Catherine Brinson¹

¹*Department of Mechanical Engineering and Materials Science, Duke University, Durham, NC, USA*

²*Department of Mechanical Engineering, California Institute of Technology, Pasadena, CA, USA*

³*Department of Electrical and Computer Engineering, Duke University, Durham, NC, USA*

Abstract

Machine learning surrogates based on neural operators have shown broad applicability in solving forward PDE problems. However, eigenvalue problems, in which an eigenparameter and one of several valid eigenmodes must be simultaneously solved, remain difficult because standard operator learning formulations assume a unique input–output map. This work demonstrates that Fourier Neural Operators (FNOs), combined with wavelet-based encodings of PDE inputs, can learn and inexpensively predict multiple eigenmodes of the elastic wave equation, corresponding to deformation modes of acoustic waves propagating through arbitrary metamaterial geometries. We provide theoretical motivation and experimental evidence for why wavelet encodings are well matched to the dual spatial–spectral structure of the FNO, enabling deterministic mode selection on both continuous and discretely valued geometries within a single model, and for why prediction accuracy varies with geometric discontinuities. These results carry broader implications for designing input encodings in other multi-mode PDE solvers based on spectral neural operators.

Keywords: Neural Operator, Metamaterials, Wavelets, Eigenmodes, Machine Learning

22	Table of Contents
----	--------------------------

23	
----	--

24 1. Introduction

25 The numerical solution of partial differential equations (PDEs) underpins modern
26 scientific computing and engineering design. Applications ranging from structural
27 mechanics and fluid dynamics to wave propagation and materials engineering rou-
28 tinely rely on computationally intensive numerical methods such as finite element
29 analysis (FEA) to obtain accurate solutions. While these methods are highly reliable,
30 their computational cost can become prohibitive in settings that require repeated
31 evaluations, such as optimization, uncertainty quantification, inverse design, and
32 real-time control.

Citation
needed

33 Recent advances in machine learning have introduced neural operators as a promis-
34 ing alternative to traditional numerical solvers. Unlike conventional neural networks
35 that learn mappings between finite-dimensional vectors, neural operators learn map-
36 pings between function spaces, allowing them to approximate entire solution operators
37 for families of PDEs. Models such as the Fourier Neural Operator (FNO) have demon-
38 strated remarkable accuracy and computational efficiency across a variety of linear
39 and nonlinear PDE problems while maintaining strong generalization capabilities
40 across discretizations and geometries. These developments have generated significant
41 interest in using neural operators as surrogate models for computational physics
42 applications.

Citation
needed

43 Despite this progress, existing neural operator architectures have primarily been
44 developed and evaluated on PDEs whose governing parameters are known a priori
45 and whose solutions are uniquely determined by the provided inputs. A large class
46 of physically important problems, however, are governed by eigenvalue PDEs, in
47 which both the solution field and one or more unknown PDE parameters must be
48 determined simultaneously. Examples include vibration analysis, quantum mechanics,
49 photonics, acoustics, and wave propagation in metamaterials. In these problems,
50 multiple valid solutions may exist for the same physical domain, corresponding to
51 different eigenvalue-eigenvector pairs. The ability to represent and distinguish among
52 these multiple solutions is essential for accurately modeling the underlying physics.

Citation
needed

53 Acoustic metamaterials provide a particularly compelling example of this chal-

Citation
needed

54 lenge. These architected materials derive their properties from geometric structure
 55 rather than composition alone, enabling engineered interactions with propagating
 56 waves. Understanding and designing such systems often requires computing multiple
 57 deformation modes associated with different resonant frequencies and wave behaviors.
 58 Traditionally, these modes are obtained through computationally expensive eigenvalue
 59 analysis using finite element methods. A neural operator capable of accurately predict-
 60 ing multiple deformation modes directly from material geometry would significantly
 61 accelerate such iterative design and analysis processes.

Citation
needed

Citation
needed

62 However, extending neural operators to eigenvalue PDEs is nontrivial. Standard
 63 operator-learning formulations implicitly assume a one-to-one mapping between in-
 64 puts and outputs, whereas eigenvalue problems admit multiple valid solutions for a
 65 given geometry. As a result, existing FNO architectures lack a mechanism for deter-
 66 ministically selecting among different eigenmodes. Furthermore, the representation of
 67 eigenvalue-related information must be compatible with the spectral architecture of
 68 the FNO, which processes information jointly in both spatial and frequency domains.

69 In this work, we introduce a framework that enables Fourier Neural Operators to
 70 solve eigenvalue PDEs associated with elastic wave propagation in acoustic metama-
 71 terials. The key idea is to augment the model inputs with wavelet-based encodings
 72 of PDE parameters that uniquely identify individual eigenmodes while remaining
 73 compatible with the FNO architecture. We demonstrate that these encodings allow
 74 a single neural operator to learn multiple valid solutions corresponding to distinct
 75 deformation modes and to reproduce those modes deterministically on demand. We
 76 further show that conventional spatial encodings and purely spectral encodings do
 77 not provide the same capability, highlighting the importance of the proposed wavelet
 78 representation. We also evaluate the same model on continuous- and discrete-valued
 79 geometries, recovering both displacement fields and eigenfrequencies, and examine
 80 how geometric discontinuities interact with the truncated spectral structure of the
 81 FNO.

82 The contributions of this paper are threefold:

- 83 1. We demonstrate that a single Fourier Neural Operator can learn multiple

eigenmodes across continuous and discrete metamaterial geometries, recovering both eigenvectors (displacement fields) and eigenvalues (eigenfrequencies).

2. We introduce wavelet encodings of modal parameters and explain theoretically why their spatial–spectral structure enables deterministic mode selection in FNOs.

3. We show that prediction accuracy degrades with geometric discontinuities and interpret this through spectral truncation, with implications for applying FNOs to problems with sharp interfaces.

Together, these results extend the applicability of neural operators beyond conventional forward PDE problems and establish a pathway toward efficient surrogate modeling of eigenvalue-driven physical systems, including acoustic metamaterials and other wave-based engineering applications.

2. Methodology

This body of work involves several different methods for generating and then training with data to enable FNO models to simulate acoustic metamaterials, a practical and nontrivial eigenvalue PDE problem. Each method is detailed in separate subsections, starting with the materials simulation side of the procedure, and then moving into the machine learning aspects. The reader is encouraged to read or skip any of these subsections depending on their interest in technical details.

2.1. Fourier Neural Operator

The principal theoretical motivation for Fourier Neural Operators is their universal approximation property over function spaces. Let $\mathcal{A}$ and $\mathcal{U}$ denote Banach spaces of input and output functions, respectively, and let

$$\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U} \tag{1}$$

107 be a continuous nonlinear operator. It has been shown that, given sufficient latent
 108 width and Fourier modes, an FNO can approximate $\mathcal{G}$ arbitrarily well on compact sub-
 109 sets of $\mathcal{A}$. Unlike conventional neural networks, which approximate finite-dimensional
 110 functions, FNOs approximate operators between infinite-dimensional function spaces
 111 by learning a sequence of global integral operators parameterized in the Fourier
 112 domain.

113 This theoretical framework assumes that both the input and output are continuous
 114 fields of infinite resolution. In practice, however, these fields are sampled on a finite
 115 computational grid. The continuous Fourier transform is therefore replaced by
 116 a discrete Fourier transform (DFT), and only a finite number of Fourier modes
 117 are retained during training. Consequently, the learned model approximates the
 118 continuous solution operator through its discrete representation while preserving the
 119 same spectral operator-learning architecture.

120 For the present work, the operator acts on discretized fields over a 32×32 pixel
 121 spatial grid. The input field consists of three channels containing encoded information
 122 for the metamaterial geometry, the stimulatory wavevectors, and the band number.
 123 The output field consists of five channels corresponding to the predicted eigenvalue, or
 124 frequency, and eigenvector, or displacement field, of the resulting deformation mode.
 125 Thus, the discretized learned operator takes the form

$$\mathcal{G}_\theta : \mathbb{R}^{32 \times 32 \times 3} \rightarrow \mathbb{R}^{32 \times 32 \times 5}. \quad (2)$$

126 The discretized FNO follows the standard lift-transform-project architecture:

- 127 1. **Lifting:** The input field is embedded into a higher-dimensional latent space
 128 through a pointwise lifting operator,

$$v_1(x) = \ell(a(x)), \quad \ell : \mathbb{R}^3 \rightarrow \mathbb{R}^{d_L}. \quad (3)$$

- 129 2. **Fourier layers:** A sequence of Fourier layers is applied to the latent represen-

130 tation. Each layer updates the hidden field according to

$$v_{j+1}(x) = \sigma \left(\mathcal{F}^{-1} [R_j(k) \mathcal{F}[v_j](k)] + W_j v_j(x) + b_j \right), \quad (4)$$

131 where $\mathcal{F}$ denotes the discrete Fourier transform, $R_j(k)$ is the learnable spectral
 132 kernel, W_j is a learnable pointwise linear transformation, b_j is a learnable bias
 133 term, and σ is the activation function. GELU was used in this work because it
 134 is smooth and differentiable.

135 3. **Projection:** The final latent representation is projected back to the desired
 136 output dimension through a pointwise projection operator,

$$u(x) = p(v_n(x)), \quad p : \mathbb{R}^{d_L} \rightarrow \mathbb{R}^5. \quad (5)$$

137 FNOs are well suited for acoustic metamaterial simulation because the underlying
 138 physics is governed by spatially distributed wave equations, where the response
 139 at one location depends on the global geometry of the unit cell and the imposed
 140 wavevector. The Fourier layers are designed to use spectral convolutions to efficiently
 141 capture long-range interactions and periodic spatial structure, while the pointwise
 142 transformations preserve local geometric information. Although FNOs are commonly
 143 used as surrogate models for fixed PDE solution operators, the same operator-learning
 144 framework can be extended to eigenvalue problems by conditioning the input on
 145 modal parameters, such as the wavevector and band index, and training the network
 146 to output both the associated eigenvalue and eigenvector field. In this formulation,
 147 the FNO learns a parameterized eigensolution operator mapping geometry and modal
 148 encodings to the frequency and deformation mode of the acoustic unit cell.

149 This process is illustrated graphically in Figure ?? . For our experiments, the
 150 NeuralOp Python package was used, which provides FNO model definitions with
 151 customizable numbers of layers, hidden channels, and input and output tensor sizes.
 152 The hyperparameter combinations explored, as well as the optimized final FNO
 153 configuration, are shown in Table ?? in Section ?? . These hyperparameters are used
 154 alongside the fixed parameters of the problem space: model height 32, model width

155 32, input channels 3, and output channels 5.

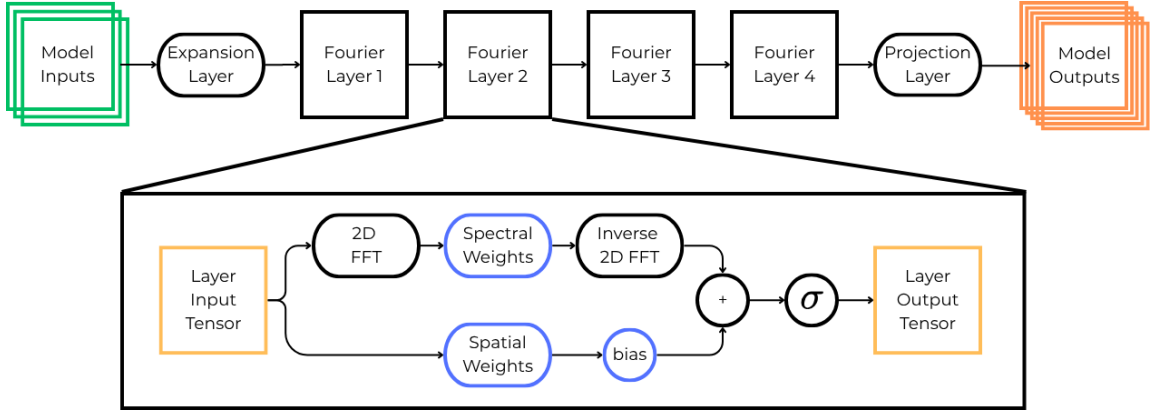


Figure 1: The FNO model architecture used in this project. The input data is first expanded according to the hidden-channels parameter, allowing multiple latent features to be represented between Fourier layers. Each Fourier layer is shown in detail in the expanded view. The data passing through a Fourier layer is processed through two learned branches: a pointwise branch, which applies a learned linear transformation in the spatial representation, and a spectral branch, which applies a fast Fourier transform (FFT), multiplies the retained Fourier modes by learned spectral weights, and then applies an inverse FFT. The outputs of both branches, along with a learnable bias term, are summed before passing through a nonlinear activation function. After passing through four Fourier layers, the latent representation is projected from the hidden-channel dimensionality to the final output dimensions.

156 2.2. Metamaterial Design Space

157 To train a surrogate model capable of predicting acoustic eigenmodes, a represen-
 158 tative design space of acoustic metamaterials must first be defined. The design space
 159 was selected to provide sufficient geometric and material complexity to demonstrate
 160 operator learning for eigenvalue PDEs while maintaining computationally tractable
 161 finite element simulations for dataset generation.

162 The dataset is restricted to two-dimensional acoustic metamaterials composed of
 163 infinitely repeating square unit cells exhibiting eight-fold symmetry. Each unit cell is
 164 discretized onto a 32×32 spatial grid and represented by three material-property
 165 channels corresponding to the normalized elastic modulus, density, and Poisson’s
 166 ratio. Material properties are normalized to the interval $[0, 1]$ to improve numerical
 167 conditioning during training, while the corresponding physical values are recovered

using the fixed material constants listed in Table ?? . The two constituent materials were chosen to represent a stiff steel alloy and a compliant polymer.

Parameter	E_{polymer}	E_{steel}	ρ_{polymer}	ρ_{steel}	ν_{polymer}	ν_{steel}
Units	Pa	Pa	kg/m ³	kg/m ³	—	—
Value	1×10^6	2×10^9	1200	8000	0.45	0.3

Table 1: Ranges for material parameters. The hard material is chosen to be a common steel alloy and the soft material a rubbery polymer.

Unit-cell geometries are synthesized by sampling a Gaussian process with a periodic covariance kernel and subsequently enforcing eight-fold symmetry. This procedure produces smoothly varying spatial material distributions while ensuring compatibility with periodically tiled square lattices. The sampled field assumes values between 0 and 1, where 0 represents pure polymer, 1 represents pure steel, and intermediate values denote an effective mixture of the two materials below the spatial resolution of the discretization.

To investigate the influence of geometry representation on model performance, two classes of unit cells were generated. The first consists of the continuous-valued Gaussian process realizations described above and is referred to as the *continuous geometry* dataset. The second is obtained by thresholding the Gaussian process samples to binary numbers, producing geometries composed exclusively of polymer and steel. This second collection is referred to as the *discrete geometry* dataset. Each representation comprises approximately one-half of the total dataset.

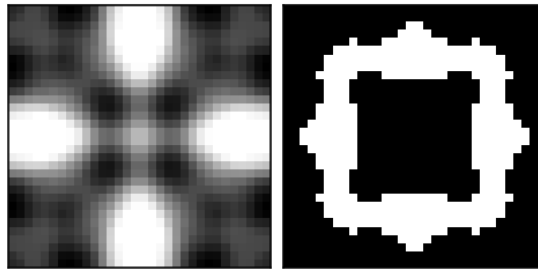


Figure 2: Example of a continuous geometry (left) and a discrete geometry (right) generated using the above process and parameters.

184 2.3. Wave Propagation Simulations

185 For each metamaterial geometry, wave propagation is simulated by solving the
 186 frequency-domain linear elastodynamic equation with finite element analysis (FEA).
 187 After spatial discretization and enforcement of Bloch periodicity, the governing
 188 problem takes the generalized eigenvalue form

$$(\mathbf{K}_r(\mathbf{k}) - \omega^2 \mathbf{M}_r(\mathbf{k})) \mathbf{u} = \mathbf{0}, \quad (6)$$

189 where ω is the angular eigenfrequency, $\mathbf{u}$ is the corresponding complex Bloch displace-
 190 ment eigenvector, and $\mathbf{k}$ is the wavevector in the irreducible Brillouin zone (IBZ). The
 191 reduced stiffness and mass matrices $\mathbf{K}_r(\mathbf{k})$ and $\mathbf{M}_r(\mathbf{k})$ are obtained from geometry-
 192 dependent global finite-element matrices $\mathbf{K}$ and $\mathbf{M}$ via a wavevector-dependent Bloch
 193 transformation matrix $\mathbf{T}(\mathbf{k})$,

$$\mathbf{K}_r(\mathbf{k}) = \mathbf{T}(\mathbf{k})^\dagger \mathbf{K} \mathbf{T}(\mathbf{k}), \quad \mathbf{M}_r(\mathbf{k}) = \mathbf{T}(\mathbf{k})^\dagger \mathbf{M} \mathbf{T}(\mathbf{k}), \quad (7)$$

194 with $(\cdot)^\dagger$ the Hermitian transpose. Here $\mathbf{T}(\mathbf{k})$ maps the full mesh degrees of freedom
 195 to an independent set while applying phase factors $e^{i\mathbf{k}\cdot\mathbf{R}}$ across periodic faces of the
 196 unit cell; $\mathbf{K}$ and $\mathbf{M}$ themselves depend only on the material field and are assembled
 197 once per geometry. These intermediate matrices are described briefly so the reader
 198 can compare the explicit FEA pipeline (Figure ??) with the Fourier neural operator
 199 architecture in Section ??; they are not inputs to the neural operator.

200 The IBZ is discretized into 25 x wave numbers and 13 y wave numbers, totaling
 201 $25 \times 13 = 325$ unique wavevectors covering the IBZ half plane. For each wavevector,
 202 the six lowest eigenfrequencies (bands) were computed using a custom FEA MATLAB
 203 script (see Section ?? for implementation details).

204 Each eigenvector $\mathbf{u}$ consists of two complex-valued displacement fields, (u_x, u_y) ,
 205 which are separated into their real and imaginary components to produce four
 206 real-valued output channels, $(u_{x,real}, u_{x,imag}, u_{y,real}, u_{y,imag})$. The corresponding eigen-
 207 frequency ω is encoded as a spatially uniform fifth channel, resulting in a target
 208 tensor of size $5 \times 32 \times 32$. Combined with the three-channel material representation

described in the previous subsection, each simulation therefore produces a supervised learning sample mapping

$$\mathbb{R}^{3 \times 32 \times 32} \rightarrow \mathbb{R}^{5 \times 32 \times 32}.$$

The overall finite element data-generation procedure is summarized in Figure ??.

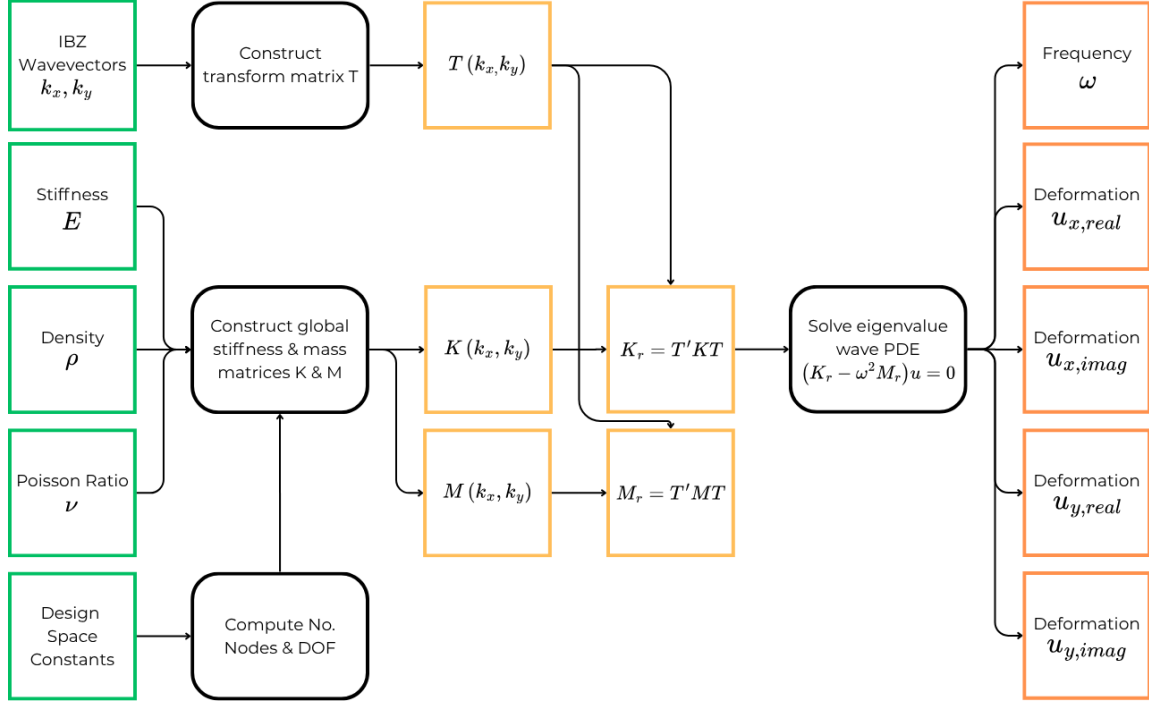


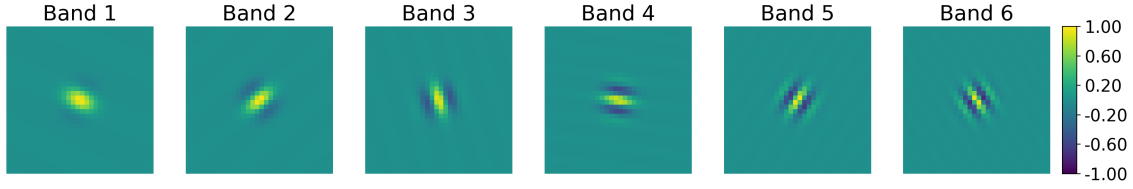
Figure 3: Flowchart of the FEA data generation method. Green squares represent varying inputs to the process, and orange squares represent computed quantities of the FEA process. Black rounded rectangles represent subroutines of the FEA process. The process starts with inputs of defined densities, stiffnesses, and Poisson ratios for each pixel of a metamaterial unit cell geometry (each a 32×32 pixel grid of values ranging from 0 to 1), along with a chosen k_x and k_y wavevector representing the stimulus to the metamaterial. A custom optimized FEA solver takes these inputs and produces intermediate matrices ($\mathbf{K}, \mathbf{M}, \mathbf{T}$), forms the reduced operators ($\mathbf{K}_r, \mathbf{M}_r$), and then solves Eq. (??) for the displacement fields (eigenvectors) and frequencies ω (eigenvalues) for the given inputs.

2.4. Input Wavelet Encoding

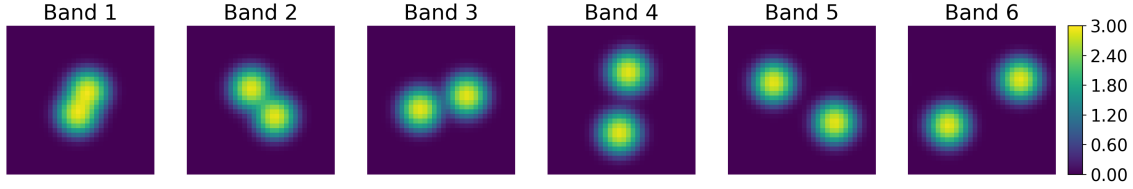
To allow the FNO model to toggle between PDE solutions for a given geometry, information must be fed to the model indicating which excitation waveform (wavevectors) and deformation mode (band) we are looking for a solution to. This

information is given in the form of a wavevector embedding, which we have found to yield superior results compared with a constant field embedding (2D matrices of the same value) or a sinusoidal embedding (2D sinusoidal fields with constants encoded as the sinusoidal frequency). A discussion on why wavelet encodings are superior is found in Section ??.

For the wavelet encodings, we use a 1D Gabor wavelet transform to map band numbers to unique wavelet images (see Algorithm ??), and a 2D Gabor wavelet transform to map x and y wavevectors to another set of unique wavelet images (see Algorithm ??). These representations show rich variation both in the spatial domain and in the Fourier domain, which allows for FNO model weights of both branches to richly interact with the wavelet representation, leading to better learning outcomes.

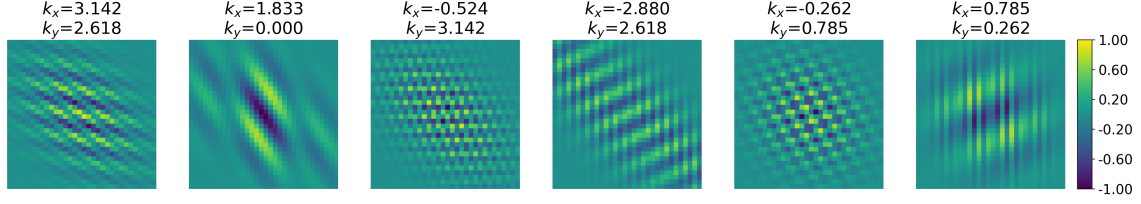


(a) The wavelet spatial encoding for band integer.

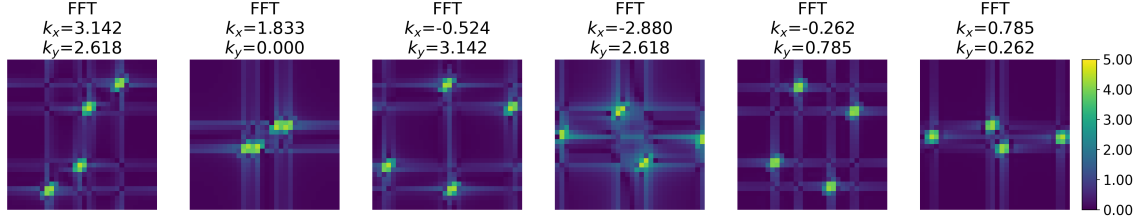


(b) The FFT of the above wavelet spatial encodings.

Figure 4: The wavelet encoding of the band integer (fig. ??) and its FFT (fig. ??). Each band corresponds to a unique wavelet with representation in both spatial and spectral domains.



(a) The wavelet spatial encoding for x and y wavevectors.



(b) The FFT of the above wavelet spatial encodings.

Figure 5: The wavelet encoding of the x and y wavevectors (??) and its FFT ???. The actual wavevectors are shown in the subplot titles. For each k_x and k_y value pair, there is a unique mapping in both the spatial and spectral domain representations.

227 When encoding continuous constants with oscillatory functions on a finite grid,
 228 it is important to verify that distinct parameter values do not alias into nearly
 229 identical patches. Otherwise the model cannot distinguish neighboring wavevectors,
 230 and mode selection collapses. We therefore checked pairwise cosine similarity of the
 231 mean-centered $\log_{10} |\text{FFT}|$ spectra of all 325 IBZ wavevector encodings produced by
 232 Algorithm ?? (Supplementary Information). The resulting similarity matrix is shown
 233 in Figure ??; the maximum off-diagonal similarity is 0.862, well below near-duplicate
 234 levels, confirming that the chosen encoding constants introduce no appreciable aliasing
 235 across the IBZ grid.

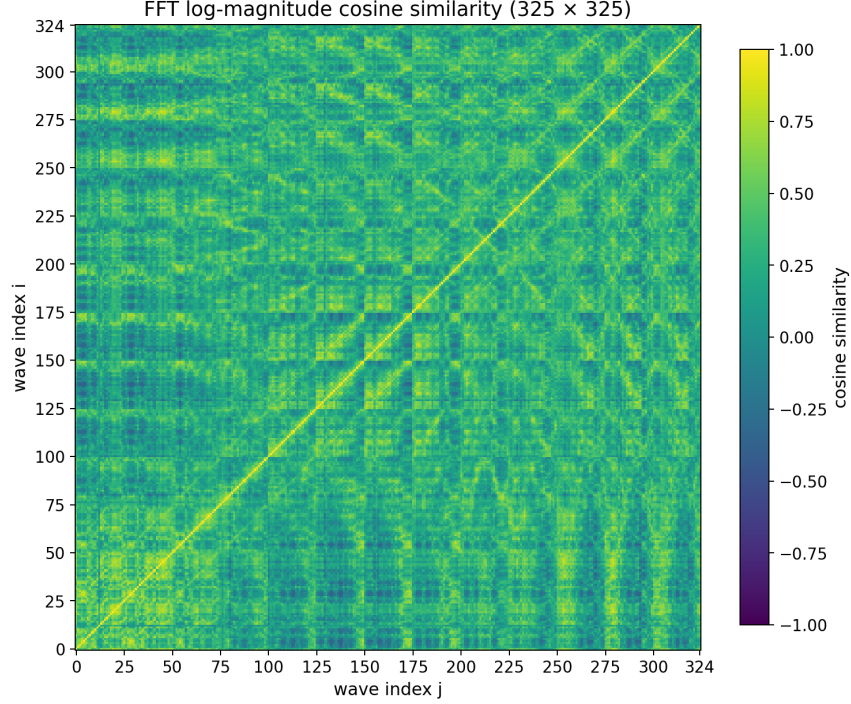


Figure 6: Pairwise cosine similarity of mean-centered $\log_{10} |\text{FFT}|$ spectra for the 325 IBZ wavevector encodings generated by Algorithm ?? . The maximum off-diagonal similarity is 0.862, indicating that distinct (k_x, k_y) pairs remain spectrally separable on the 32×32 grid.

2.5. Dataset Construction

Our full dataset contains 24000 geometries, randomly generated as described in prior subsections. Half are discrete valued with only 0s or 1s in each pixel location representing one of the two materials, while the other half are continuous valued with any value between 0 and 1 inclusive at each pixel location, representing blended materials at each pixel location. Each geometry has 6 possible bands each with 325 possible wavevectors (representing a mesh grid the IBZ half plane) per band, resulting in $24000 \times 6 \times 325 = 46,800,000$ sample pairs of inputs with shape $(3 \times 32 \times 32)$ and outputs of shape $(5 \times 32 \times 32)$. The data is stored with float16 datatype format in PyTorch tensors.

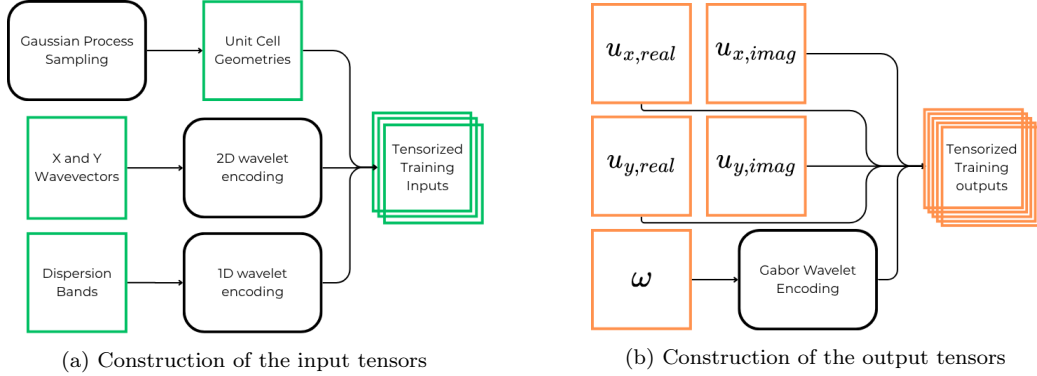


Figure 7: Subfigure ?? and ?? shows the process of input data from subsection ?? and output data from subsection ?? being tensorized for ingestion to the FNO model respectively. Note the wavelet embedding steps on fundamentally scalar quantities.

2.6. Training Procedure

The Fourier Neural Operator was trained using supervised learning on paired inputs and outputs described in Section ?. Each training sample consists of a $3 \times 32 \times 32$ input tensor (geometry and wavelet embeddings) and a $5 \times 32 \times 32$ output tensor (displacement field components and eigenfrequency).

6 loss functions were trialed as performance evaluation criterion: mean absolute error (MAE), mean squared error (MSE), normalized MAE, normalized MSE, Huber (smooth L1), and structural similarity index (SSI). Losses were evaluated and aggregated across all 5 output channels. The best performing loss criterion is the Huber loss and is shown below. Details on the other loss functions trialed can be found in Supplemental Information.

$$\mathcal{L}_{\text{Huber}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^4 \begin{cases} \frac{1}{2} (\hat{u}_c(i, j) - u_c(i, j))^2, & |\hat{u}_c(i, j) - u_c(i, j)| \leq \delta, \\ \delta \left(|\hat{u}_c(i, j) - u_c(i, j)| - \frac{\delta}{2} \right), & |\hat{u}_c(i, j) - u_c(i, j)| > \delta. \end{cases}$$

where $H = W = 32$ (pixels) and $c \in \{0, 1, 2, 3, 4\}$ corresponds to the 5 output channels: eigenfrequency, followed by real and imaginary components of x and y displacements.

260 For training the FNO model, combinations of model layers, hidden channels,
261 activation function, loss function, optimizer, and training hyperparameters were
262 trialed. The ADAMW optimizer was found to achieve the best performance compared
263 with other flavors of momentum based optimizers and SGD. The model and training
264 hyperparameters tuned are showed in table ?? in ??, with the best performing
265 combination italicized.

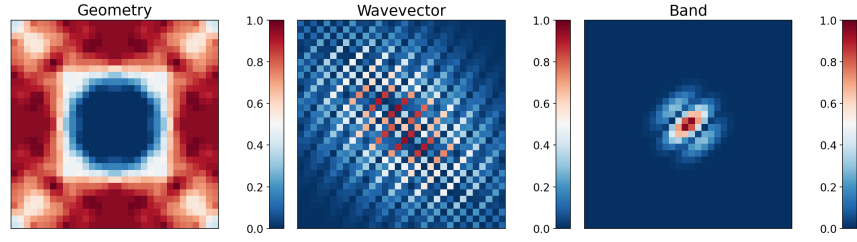
266 The best combination for model hyperparameters was found to be 4 Fourier layers,
267 128 hidden channels, and GELU nonlinear activation function. At this layer count
268 and hidden channel count, performance was asymptotic and greater amounts of either
269 would drive the model further into the overfit region as well as add on additional
270 training time. The best combination for trianing hyperparameters were the Huber loss,
271 which is to use a which would was found to be 12 epochs, Different hyperparameters
272 of FNO model tried in combinations. 4 Fourier layers, with 64 hidden channels and
273 GELU activation function were found to be where performance started to asymptote
274 with increasing model complexity, and so it was not worth the computational costs
275 in training to use more layers or hidden channels. These are the final combination of
276 model hyperparameters from which all data presented in this article is generated.

277 It was found that final model performance did not appreciably change for changes
278 in step size or batch size fairly early on, so those two parameters were left fixed at 4
279 and 256 respectively for the remainder of the training runs. Learning rate and epochs
280 to convergence as expected, had an inverse correlation. The higher the learning rate,
281 the faster convergence happened, but there are two problems with high learning
282 rates. The first is that the model may arrive at a more suboptimal combination of
283 weights if the training rate is too high, because the movements in weight space during
284 each backward pass remain too large to arrive close in weight space to where the
285 loss minima would occur. The second, more interesting phenomena, is that if the
286 learning rate is set higher than $\approx 10^{-2}$, there is instability introduced in the training,
287 whereby at certain epochs, the loss would spike, and never come back down, usually
288 plateauing, in all later epochs. This sort of spike is documented in other applications
289 of ADAM family optimizers indicating the model is pushed into a region of weight
290 space where the loss was flat for movements in any direction.

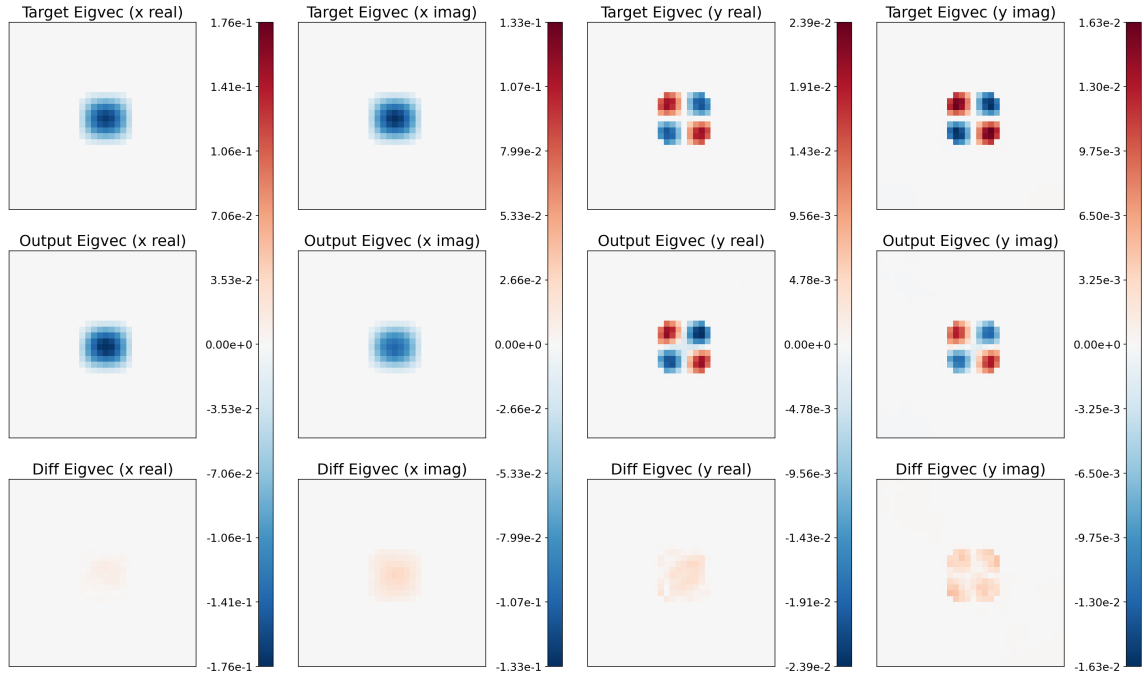
3. Results and Discussion

3.1. Displacement Field Prediction

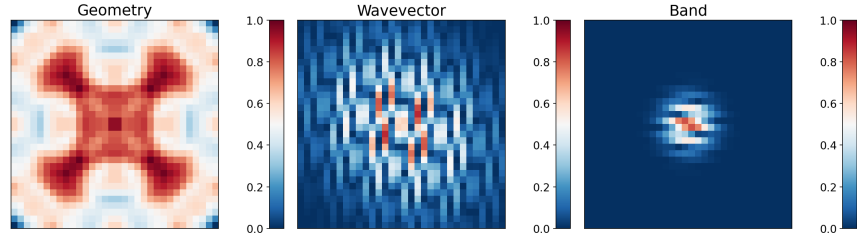
We next evaluate the trained Fourier Neural Operator on its predictions of the Bloch displacement fields (u_x, u_y) . For each unseen test sample, the prediction error is measured by the normalized mean absolute error (NMAE) over the four real-valued displacement channels, $(u_{x,\text{real}}, u_{x,\text{imag}}, u_{y,\text{real}}, u_{y,\text{imag}})$. Ranking samples by this NMAE defines a performance distribution from which we select representative cases at the 25th, 50th, and 75th percentiles, corresponding to relatively weak, median, and relatively strong predictions. The same percentile comparison is shown for both the binarized and continuous material datasets, allowing direct visual assessment of how well the model recovers the spatial structure of the displacement modes across contrasting geometry regimes.



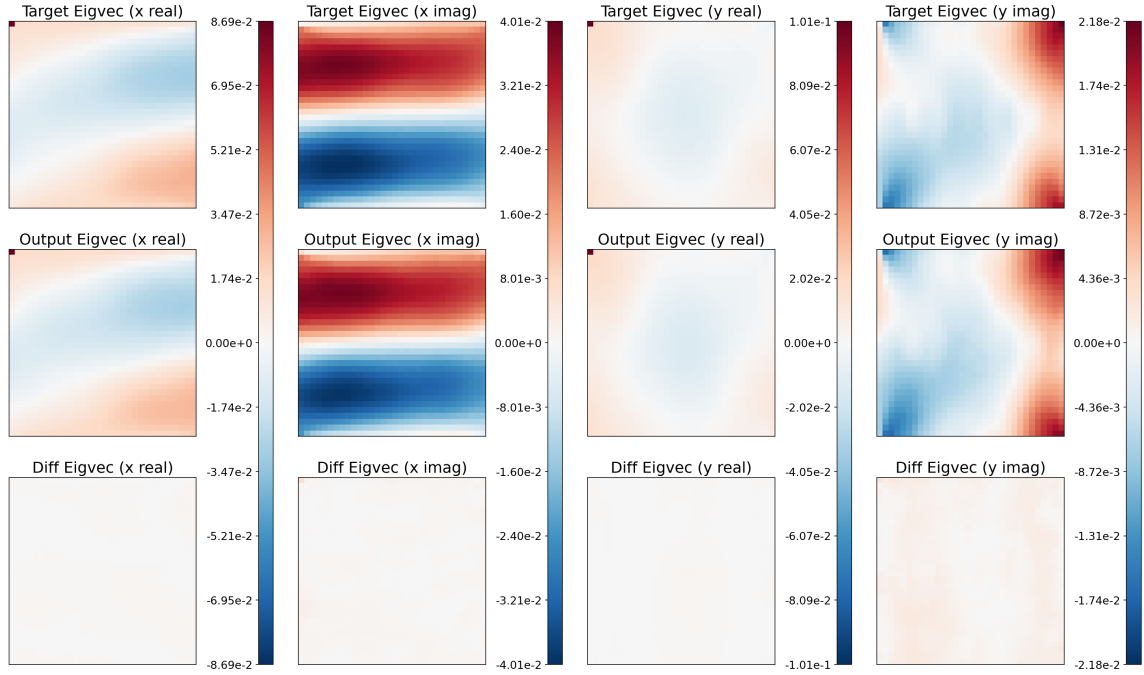
(a) Input sample at the 25th percentile of performance for unseen continuous valued geometries.



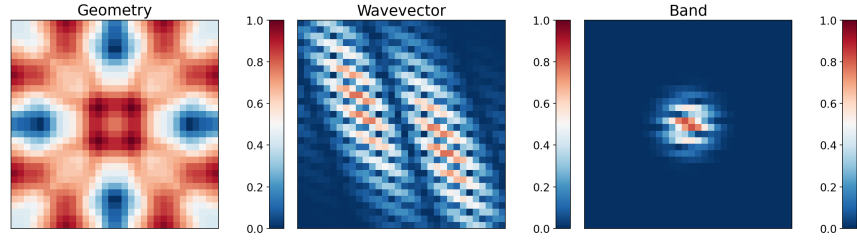
(b) Output predictions at the 25th percentile of performance for unseen continuous valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. At the 25th percentile, prediction outcomes are already fairly good, with accurate representations of scale and structure.



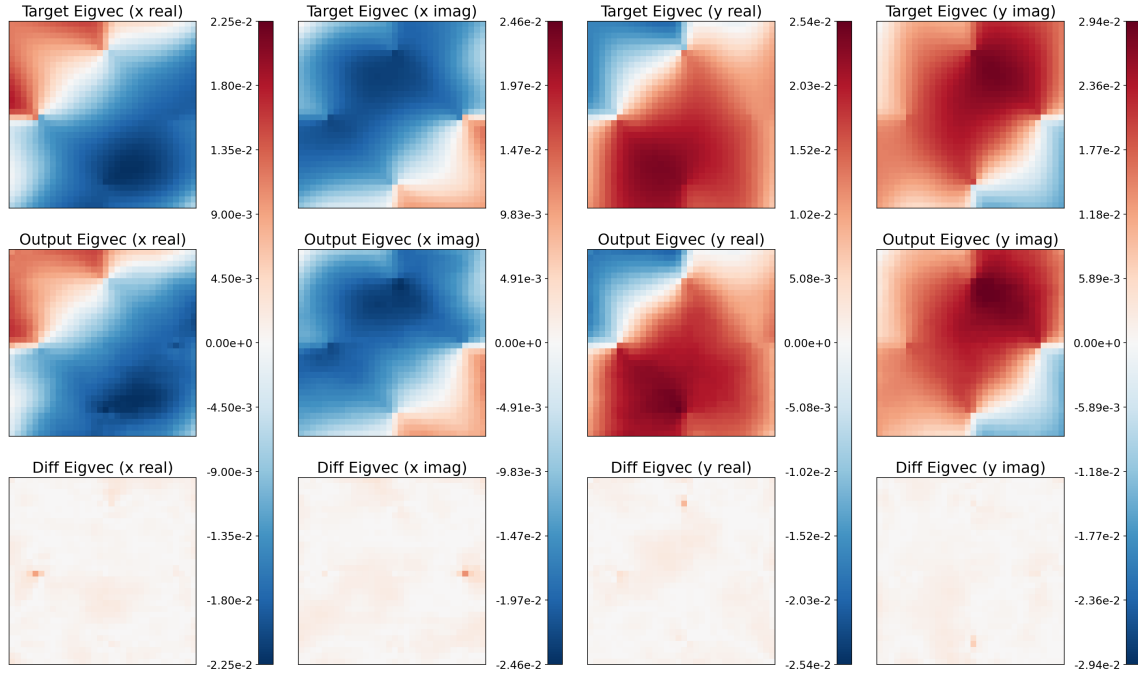
(a) Input sample at the 50th percentile of performance for unseen continuous valued geometries.



(b) Output predictions at the 50th percentile of performance for unseen continuous valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. Predictions are able to consistently capture complex structures without graininess.

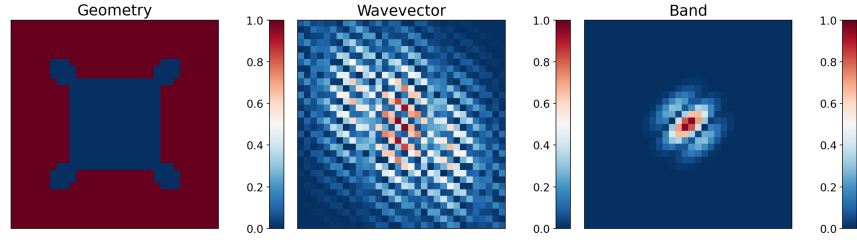


(a) Input sample at the 75th percentile of performance for unseen continuous valued geometries.

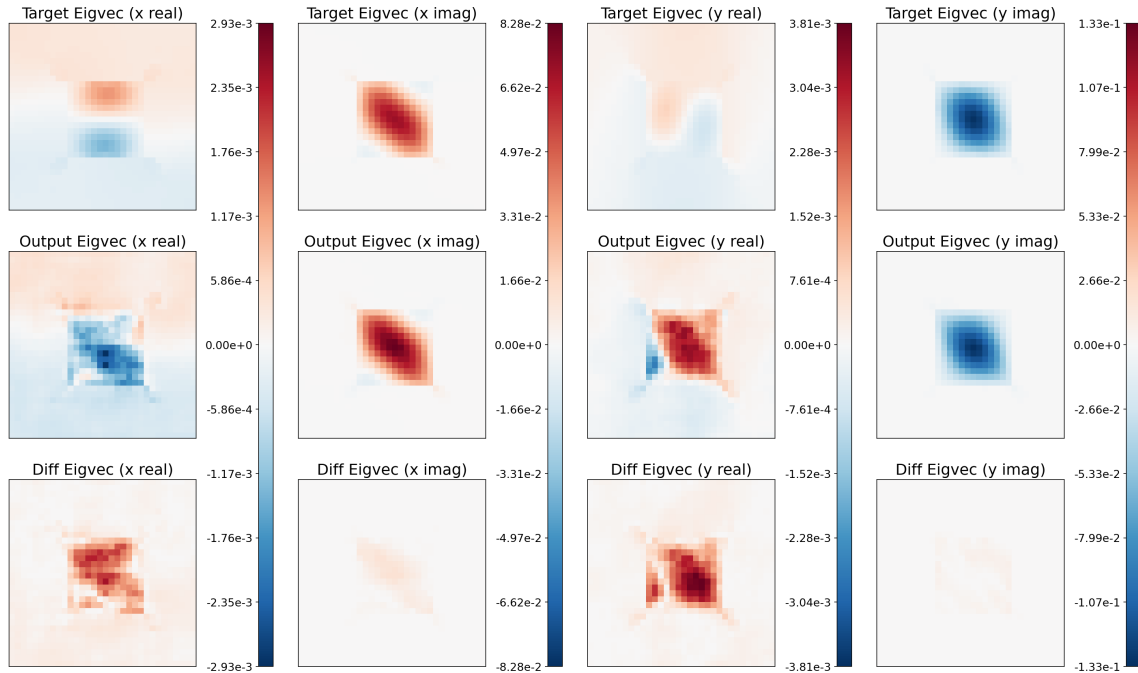


(b) Output predictions at the 75th percentile of performance for unseen continuous valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. At this point, the target and prediction are virtually indistinguishable by eye.

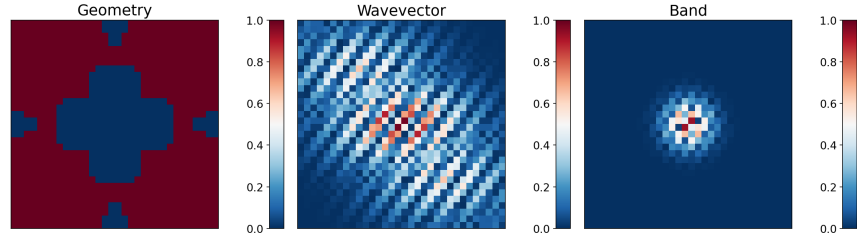
304 These results demonstrate that with wavelet encodings, the FNO model performs
 305 well on continuous geometries, faithfully reproducing the displacement fields in most
 306 cases. Model predictions in the top three quartiles match targets well in structure
 307 and scale, with decreasing average pixel relative errors in the scale of 10^{-2} smoothly
 308 decreasing down to 10^{-4} with increasing percentile performance. (See Figure ?? for
 309 more detailed statistics.)



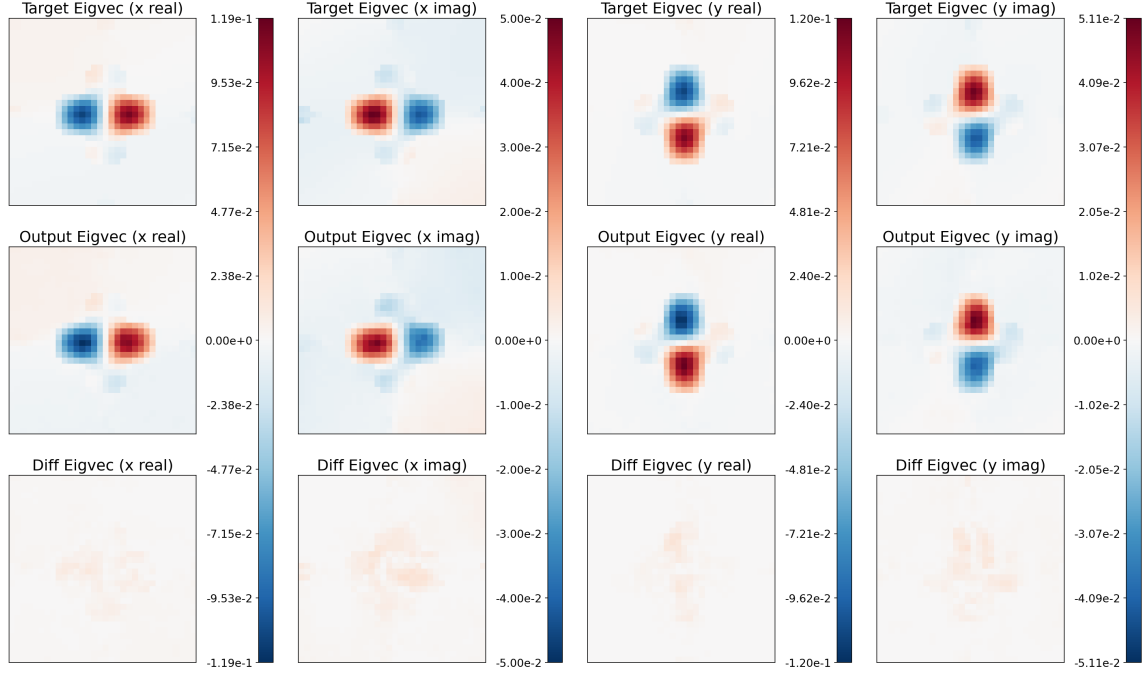
(a) Input sample at the 25th percentile of performance for unseen discrete valued geometries.



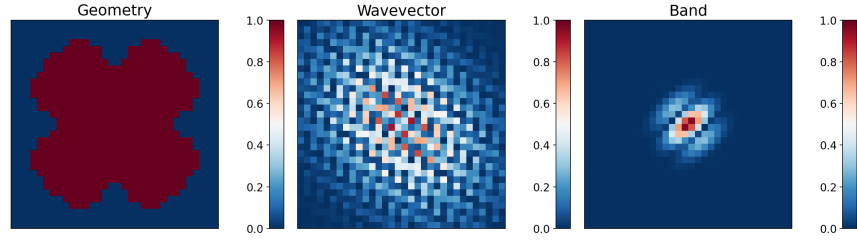
(b) Output predictions at the 25th percentile of performance for unseen discrete valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. Due to the nature of the loss function training, for this particular sample, the model would have prioritized the two imaginary displacement fields, because they are 1 to 2 orders of magnitude larger in scale compared to the two real displacement fields. In other words, absolute error is low at this point, but relative error is not for some channels of small scale than other channels. (Note that the colorbars for each column are independently scaled.)



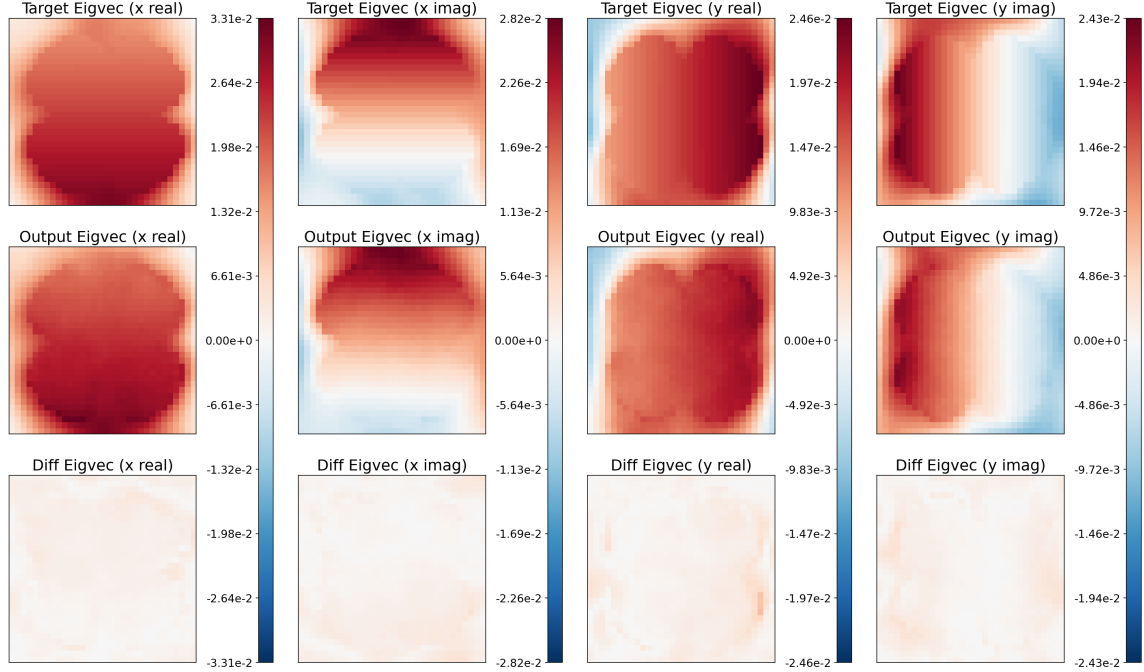
(a) Input sample at the 50th percentile of performance for unseen discrete valued geometries.



(b) Output predictions at the 50th percentile of performance for unseen discrete valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. At this point, absolute and relative errors are low and there is high agreement in structural and scale between targets and predictions.



(a) Input sample at the 75th percentile of performance for unseen discrete valued geometries.



(b) Output predictions at the 75th percentile of performance for unseen discrete valued geometries. The first row are target fields, followed by prediction fields on the second row, and difference fields on the third. At this point, disagreements between targets and predictions become very hard to see by eye.

Like with the continuous geometry case, these results demonstrate that with wavelet encodings, the FNO model performs well on discrete geometries, faithfully reproducing the displacement fields in most cases. Model predictions in the top three quartiles match targets well in structure and scale, with decreasing average pixel relative errors in the scale of 10^{-2} smoothly decreasing down to 10^{-4} with increasing percentile performance. (See Figure ?? for more detailed statistics.)

3.1.3. Continuous Versus Discrete Performance

The model being able to simultaneously predict well on both continuous and discrete geometries is a non-trivial and important takeaway for the capabilities of FNOs with paired with wavelet encodings. Prior encoding strategies of constant fields and sinusoidal fields delivered far worse results with grainy structures or mode collapse issues because conditioning constants (wavevector and band) information could not propagate well through the spectral branch of the FNO layers. The performance over all unseen samples in the test set are shown in ??.

The poorer performance on binarized geometries is consistent with basic operator-learning theory. Continuous material fields lie in smoother function classes, whereas binary two-phase designs contain jump discontinuities with nonexistent derivatives at material interfaces. Since an FNO approximates the FEA operator $\mathcal{G} : \mathcal{U} \rightarrow \mathcal{V}$ through truncated Fourier representations, it is naturally advantaged when both inputs and outputs are comparatively smooth. Discrete geometries thus perform worse due to sharp boundaries, localized gradients, and broadband high-wavenumber content that are poorly captured on a fixed 32×32 grid and are mismatched to smooth wavelet encodings. Additionally, because continuous geometries have a greater tendency to have large sections of fields with zero or low displacements, a small spike is seen in relative error performance due to the near zero displacements which results in a small denominator.

These results indicate two important takeaways. The first is that it is possible for FNOs with wavelet encodings to learn multiple deformation modes for each geometry which the user can toggle between arbitrarily by setting the wavevector and band inputs. This is to our knowledge the first demonstration of FNOs on solving eigenvalue PDEs. The second, is that it is possible for that same model to simultaneously learn very different distributions of geometries (continuous valued and discrete valued), indicating it might be possible to distill out a generalized understanding of the PDE itself independent of geometry distributions and statistical artifacts. This is a possible subject for future work.

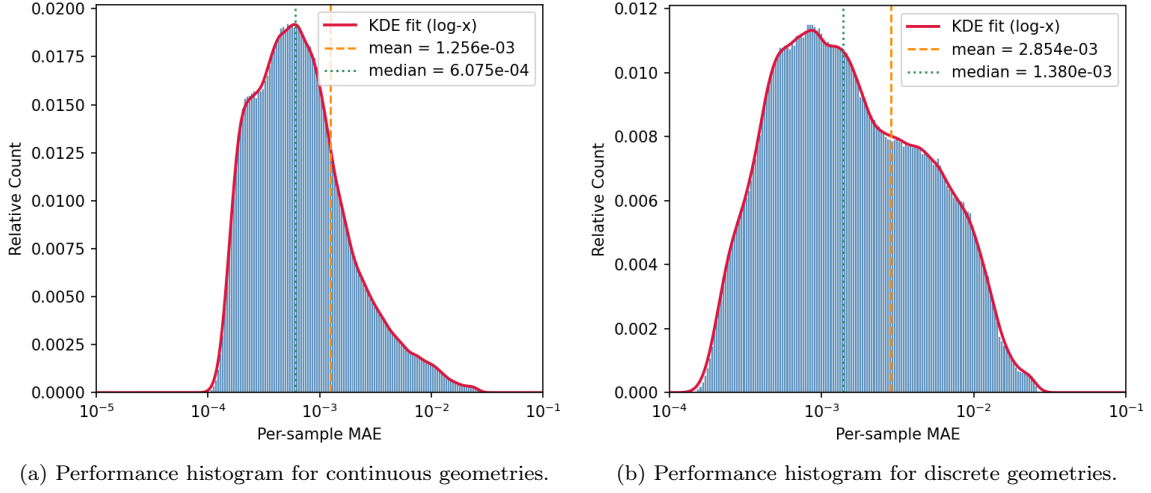


Figure 14: Comparison of relative prediction error distributions for continuous-valued and discrete metamaterial geometries for the same model. While there are some tail outliers, most samples fall within the range of 10^{-2} to 10^{-4} , indicating that the same model is able to learn continuous and discrete geometry cases.

3.2. Dispersion Band Reconstruction

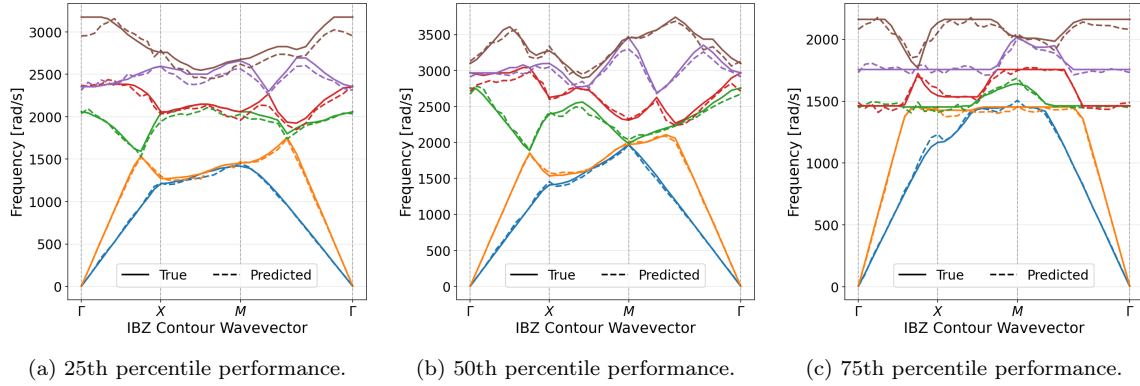


Figure 15: Predicted and ground-truth dispersion bands for representative unseen continuous valued geometries spanning the 25th, 50th, and 75th percentiles of prediction performance.

These plots represent the eigenfrequency predictions of the model across all 325 wavevectors forming the half plane for continuous valued geometries, downselected to be the horizontal, vertical, and diagonal traversal of the IBZ. Note that because the encoding of the eigenfrequency into a field has a log operation (to allow the model

to learn relationships at multiple scales), errors tend to be relative to the scale of the target, and thus higher bands may seem to be more deviant on a linear scale plot. Overall, the model performance is quite good at predictions in the encoded eigenfrequency, showing an average of $< 1\%$ prediction error across all unseen samples.

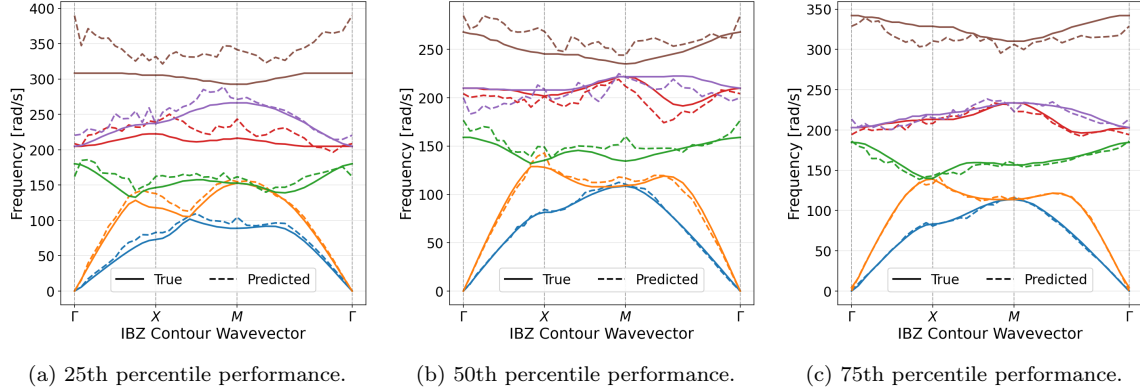


Figure 16: Predicted and ground-truth dispersion bands for representative unseen discrete valued geometries spanning the 25th, 50th, and 75th percentiles of prediction performance.

Like with the continuous geometry figures, these plots represent the eigenfrequency predictions of the model across all 325 wavevectors forming the half plane for discrete valued geometries, downselected to be the horizontal, vertical, and diagonal traversal of the IBZ. Overall, the model performance is quite good at predictions in the encoded eigenfrequency, showing an average of $< 1\%$ prediction error across all unseen samples.

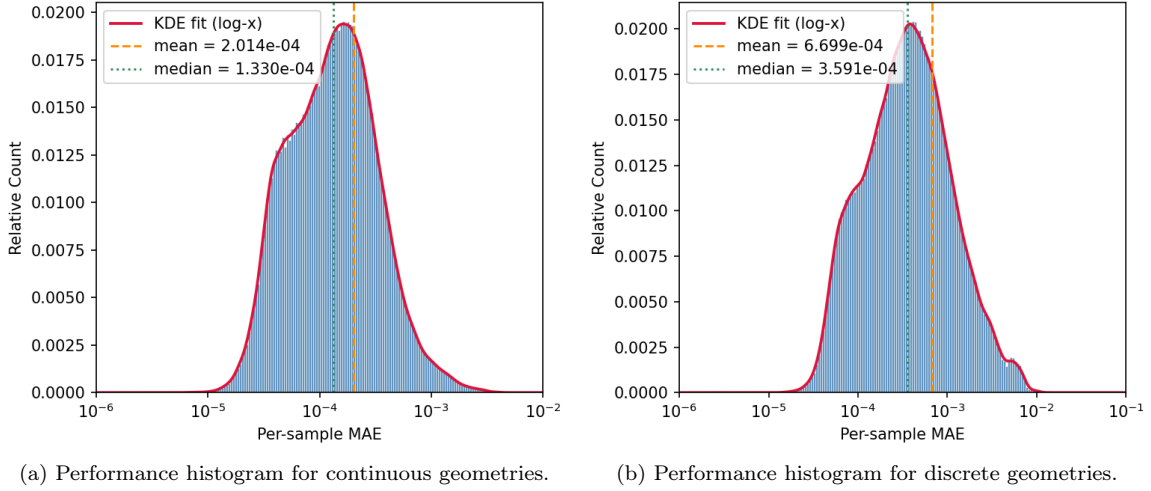


Figure 17: Comparison of relative prediction error distributions for continuous-valued and discrete metamaterial geometries for the same model. While there are some tail outliers, most samples fall within the range of 10^{-2} to 10^{-4} , indicating that the same model is able to learn continuous and discrete geometry cases.

3.3. Dependence of Prediction Error on Band and Boundary Length

The preceding subsections showed that, although the same wavelet-conditioned FNO can predict displacement fields for both continuous and discrete unit cells, discrete geometries are systematically more difficult, with absolute and relative errors larger (Figures ??, ??), and the harder quartiles of the discrete test set degrading more visibly than their continuous counterparts. This gap is expected from the architecture of the FNO itself. Each Fourier layer retains only a truncated set of spectral modes after the discrete FFT. Smooth continuous material fields, and the comparatively smooth displacement modes they induce, concentrate their energy in a modest number of low-to-moderate wavenumbers and are therefore well represented by that truncated Fourier basis. Binary geometries, by contrast, contain jump discontinuities at material interfaces. Under an FFT, such discontinuities produce a broadband spectrum whose coefficients decay slowly with wavenumber, so a substantial fraction of the high-frequency content needed to resolve sharp phase boundaries lies outside the retained modes. The spectral branch therefore sees a degraded, band-limited version of the interface, yielding Gibbs-type ringing or blur and a harder operator-learning problem

precisely when geometric discontinuity is more severe. If this spectral-truncation explanation is correct, then even among discrete geometries the prediction error should increase with the boundary interface length. To test this empirically, we define the *boundary length* of each binarized unit cell as the number of interior four-connected pixel edges separating a 0-valued pixel from a 1-valued pixel. This scalar is a discrete measure of interface complexity. For each of the 1000 unseen discrete test geometries, we compute the mean absolute error (MAE) of the predicted displacement channels, averaged over all 325 IBZ wavevectors, separately for each of the six retained eigenbands. The resulting relationships are shown in Figure ??.

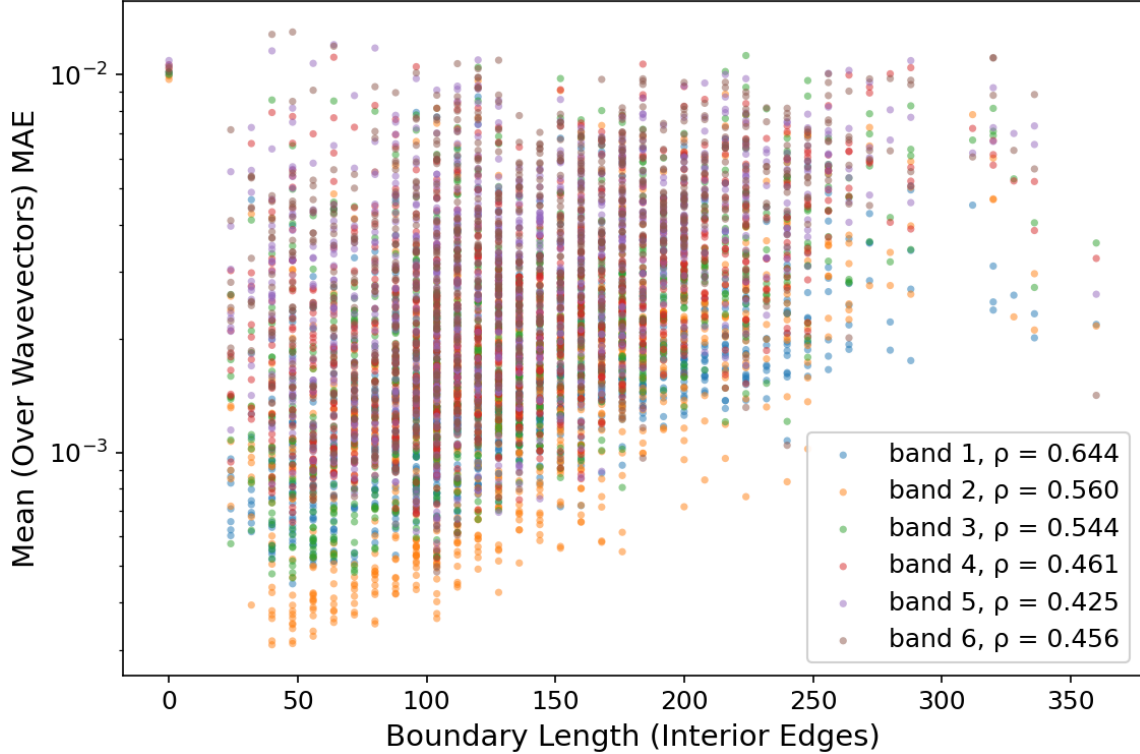


Figure 18: Mean displacement MAE versus interface boundary length, separated by color for each of the six eigenbands on the discrete test set. Points correspond to average performance on unit-cell geometries. Legend values ρ are Spearman rank correlations. A positive trend appears for every band, consistent with degraded FNO accuracy as the total length of 0/1 material interfaces increases.

Two complementary trends are apparent. First, for every band there is a positive

monotonic association between boundary length and MAE, with Spearman correlations ranging from $\rho = 0.644$ for band 1 to $\rho \approx 0.42$ – 0.56 for the higher bands. Geometries with short interfaces are concentrated at lower absolute error, while geometries with long, meandering phase boundaries systematically occupy the upper portion of the cloud. This supports the interpretation that the hard cases for the learned operator are those for which the truncated Fourier representation must resolve more extensive sharp material transitions on the fixed 32×32 grid. Second, absolute error itself increases with band index. Averaged over the test geometries, the mean wavevector-averaged MAE rises from approximately 1.6×10^{-3} for band 1 to approximately 3.8×10^{-3} for band 6. Higher bands correspond to more rapidly oscillating Bloch modes that concentrate energy nearer material interfaces and therefore inherit more of the geometric difficulty encoded by boundary length. Together with the continuous-versus-discrete comparisons above, Figure ?? indicates that prediction difficulty on discrete designs is organized both by band (modal complexity) and by interface measure (geometric complexity), as expected when a spectral surrogate retains only a portion of the broadband Fourier content generated by discontinuities.

3.4. Band and Wavevector Encoding

For the FNO to ingest the conditioning constants of the eigenvalue PDE in equation ??, namely the wavevectors k_x , k_y and the band index b that selects which eigenmode to return, these scalars must be represented as matrices or tensors compatible with the existing channel wise input structure. Alternatives that inject scalars through specialized pathways would require heavy architectural alterations, which are hard to justify given that FNO theory suggests eigenvalue PDEs should be solvable with existing hyperparameter combinations. We therefore compared three strategies for encoding these constants as input fields: constant valued fields, spatial sinusoids, and the Gabor wavelet encoding described in Section ??.

The constant field representation is the naive default: k_x , k_y , and b are each broadcast to a constant $S \times S$ channel with $S = 32$. For the wavevectors, whose physical values lie in $[-\pi, \pi]$, these constants were normalized to the interval $[-1, 1]$ before broadcasting. Band indices $\{1, 2, 3, 4, 5, 6\}$ were likewise normalized to the

range $[0.1, 0.6]$. This representation fails by mode collapse, with the FNO outputting near constant fields regardless of conditioning inputs. The cause is spectral: the FFT of a constant field is zero everywhere except at the DC coefficient, so the frequency domain branch of each Fourier layer receives no information to propagate or train on past the first layer. The Fourier branch therefore cannot contribute to the prediction, leaving only the shallow spatial domain branch, which behaves like a shallow CNN and is unable to capture the complexity of the eigenvalue problem.

A spatial sinusoidal encoding restores unique and nondegenerate representations in the spectral domain and allows the FNO to learn the problem. On pixel coordinates $(x, y) \in \{0, \dots, S - 1\}$, wavevector and band were encoded as

$$I_k(x, y) = \cos\left(\frac{k_x}{S}x + \frac{k_y}{S}y\right), \quad k_x, k_y \in [-\pi, \pi].$$

$$I_b(x, y) = \frac{1}{2} \left[\cos\left(\frac{2\pi bx}{S}\right) + \cos\left(\frac{2\pi by}{S}\right) \right], \quad b \in \{1, 2, 3, 4, 5, 6\}.$$

However, because a sinusoid is spectrally sparse, with energy concentrated at only a few wavenumbers, the Fourier branch remains weakly utilized, and predictions exhibit a persistent grainy, under resolved structure relative to ground truth. Increasing model depth and width did not remove this graininess, indicating the bottleneck was the encoding rather than model capacity.

The Gabor wavelet encoding resolves this by design. Wavelets are constructed to trade certainty between the spatial and spectral domains: neither representation is infinitely sharp, but both remain informationally rich. As a result, the conditioning inputs populate both branches of each Fourier layer with usable structure, both branches can train and contribute, and the model yields the accurate, well resolved predictions reported above while enabling deterministic mode selection. These observations support the broader conclusion that input encodings should be matched to the spatial and spectral structure of the operator: encodings that populate only one domain underuse the FNO’s Fourier branch, whereas wavelet encodings exercise both.

441 4. Conclusions

442 This work demonstrates that a single Fourier Neural Operator can learn multiple
443 eigenmodes of the elastic wave equation across both continuous and discrete metama-
444 terial geometries, recovering the eigenvectors as complex displacement fields and the
445 eigenvalues as eigenfrequencies. For unseen geometries of both types, displacement
446 predictions achieve relative errors typically between 10^{-2} and 10^{-4} and dispersion
447 reconstructions average below 1% error, showing that one model can span two very
448 different geometry distributions and output predictions of multiple orders of magni-
449 tude while allowing the user to select any eigenmode deterministically and repeatably
450 through arbitrary choice of conditioning inputs. This capability crucially rests on how
451 the conditioning constants are encoded. Constant field encodings collapse to a single
452 DC coefficient in the spectral domain, preventing the Fourier branch of the FNO from
453 propagating information and causing mode collapse. Sinusoidal encodings are able to
454 train meaningfully but yield grainy, under resolved predictions due to spectral model
455 sparsity. Gabor wavelet encodings, by contrast, trade decreasing certainty in the
456 spatial and spectral representations to maintain informationally rich representations
457 in both domains, allowing both branches of each Fourier layer to richly interact with
458 the data, leading to superior learning speed and prediction performance.

459 The same spectral perspective explains the limits we observe. Prediction error
460 grows with the interface length of discrete geometries and with band index, consistent
461 with the truncated Fourier representation struggling to resolve the broadband content
462 generated by sharp material interfaces and rapidly oscillating modes. Practitioners
463 applying FNOs to interface rich problems should therefore expect accuracy to track
464 geometric discontinuity. Even so, the trained model is lightweight, roughly 2 GB at
465 float16 precision, and evaluates in about 1 ms per sample on CPU, compared with
466 about 1 s per sample for the corresponding MATLAB FEA solve on CPU. This makes
467 the FNO a compelling surrogate for iterative design and analysis in metamaterials
468 and other fields governed by eigenvalue PDEs. Future work includes generalization
469 to other resolutions, physics informed losses, and additional conditioning parameters
470 such as material constants, expanding the accessible problem and solution space.

471 **AI Usage Disclosure**

472 Generative AI tools were used to assist with coding of experiments, translating
473 MATLAB code to Python, and automating training pipelines. AI tools were also used
474 for light editing of the manuscript. All scientific claims, results, and final wording
475 were reviewed and approved by the authors.

476 **Declaration of Competing Interest**

477 The authors declare that they have no known competing financial interests or
478 personal relationships that could have appeared to influence the work reported in
479 this paper.

480 **CRedit authorship contribution statement**

481 **Han Zhang:** Conceptualization, Data curation, Formal analysis, Investigation,
482 Methodology, Software, Validation, Visualization, Writing – original draft, Writing –
483 review and editing.

484 **Alexander Ogren:** Data curation, Software.

485 **Cynthia Rudin:** Supervision, Writing – review and editing.

486 **Johann Guilleminot:** Formal analysis, Methodology, Supervision, Writing – review
487 and editing.

488 **L. Catherine Brinson:** Conceptualization, Funding acquisition, Methodology,
489 Project administration, Resources, Software, Supervision, Visualization, Writing –
490 review and editing.

491 **Data availability**

492 Python code and experiments can be found at [https://github.com/trutheresy/](https://github.com/trutheresy/NO-2D-Metamaterials)
493 [NO-2D-Metamaterials](https://github.com/trutheresy/NO-2D-Metamaterials).

494 MATLAB FEA code can be found at [https://github.com/aco8ogren/2D-dispersion.](https://github.com/aco8ogren/2D-dispersion.git)
495 [git](https://github.com/aco8ogren/2D-dispersion.git).

496 **Acknowledgements**

497 This work was supported by the NSERC Postgraduate Scholarship – Doctoral
498 (PGSD-599307-2025) and the NSF AI for Understanding and Designing Materials
499 Research Traineeship (DGE-2022040).

500 **References**

501 S1. Supplementary Information

502 S1.1. Model and Training Hyperparameters

Hyperparameter	Candidate Settings			
Fourier Layers	4	6	8	12
Hidden Channels	32	64	128	256
Activation Function	ReLU	GELU	tanh	Sigmoid
Loss Criterion	L1	L2	Huber	SSIM
Epochs	8	12	16	20
Learning Rate	10^{-4}	10^{-3}	10^{-2}	10^{-1}
Weight Decay	0	10^{-4}	10^{-3}	10^{-2}
Gamma	0	0.01	0.1	0.5
Step Size	0	1	4	10
Batch Size	64	128	256	512
Data Precision	F8 E4M3	F8 E5M2	F16	F32

Table S1: Combinations tried in grid search for optimal model and training hyperparameters.

503 S1.2. Loss Criterion

504 In the exploration of optimal training protocols for the FNO model, other loss
505 functions were trialed as training criterion: mean absolute error (MAE), mean squared
506 error (MSE), normalized MAE, normalized MSE, Huber (smooth L1), and structural
507 similarity index (SSI). For purposes of comparison, the validation criterion was chosen
508 to be MAE and MSE. Broadly speaking, models that performed the best in terms of
509 MSE also performed the best in MAE. The training criterion that yielded the lowest
510 MAE and MSE loss was the Huber loss. All criterion followed a learning rate of
511 2×10^{-3} with a decay factor per epoch of 0.9. The mathematical definitions for each
512 area as follows. Losses were evaluated and aggregated across all 5 output channels.

- 513 • The mean absolute error (MAE) loss is defined as

$$\mathcal{L}_{\text{MAE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^4 |\hat{u}_c(i, j) - u_c(i, j)|$$

Experimentally, models trained exclusively with MAE produced predictions with sharper interfaces and better-preserved boundary features than those trained solely with MSE. This behavior is consistent with the linear penalty of MAE, which does not disproportionately penalize isolated large residuals near discontinuities. Despite the improved qualitative sharpness, MAE training converged to higher validation MAE and MSE than Huber loss, indicating that the improved preservation of fine structural features came at the expense of overall numerical accuracy. This observation also motivated the hybrid training schedule in which optimization began with MAE before transitioning to MSE, combining the edge-preserving characteristics of MAE with the stronger refinement of large residuals provided by MSE.

- The mean squared error (MSE) loss is defined as

$$\mathcal{L}_{\text{MSE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^4 (\hat{u}_c(i, j) - u_c(i, j))^2$$

The quadratic penalty of MSE assigns increasingly large gradients to larger residuals, causing the optimizer to preferentially reduce regions with the greatest numerical error. As a consequence, the loss favors smooth predictions that minimize the overall energy of the residual rather than preserving sharp transitions. Experimentally, models trained exclusively with MSE produced outputs that reproduced the correct magnitude and large-scale structure of the solution, but exhibited diffuse interfaces and blurred boundaries. Fine-scale features were often replaced by low-amplitude, grainy artifacts, consistent with the tendency of MSE to average over uncertain or discontinuous regions in order to minimize the global squared error.

- A hybrid MAE–MSE training schedule was also evaluated, where the model was optimized using MAE for the first half of the training epochs before switching to MSE for the remaining half. This training strategy approximates the behavior of the Huber loss over the course of optimization. During the early stages of

training, prediction errors are typically large, making the linear penalty of MAE advantageous for preserving sharp interfaces while preventing large residuals from dominating the optimization. As training progresses and the average residual decreases, switching to MSE increases the emphasis on refining the remaining prediction errors and improving overall numerical accuracy.

Experimentally, this training schedule consistently outperformed training with either MAE or MSE alone for the same total number of epochs. The resulting predictions retained substantially sharper boundaries than models trained exclusively with MSE, while achieving lower validation MAE and MSE than models trained exclusively with MAE. Among the non-Huber training strategies investigated, the hybrid schedule produced the best overall balance between qualitative feature preservation and quantitative accuracy, converging to within approximately 10% of the validation error achieved by Huber loss. Although less adaptive than Huber, which transitions between MAE- and MSE-like behavior on a per-residual basis, the epoch-wise transition captures much of the same optimization benefit.

- The structural similarity index measure (SSIM) is defined as

$$\mathcal{L}_{\text{SSIM}} = 1 - \frac{1}{5} \sum_{c=0}^4 \frac{(2\mu_{u_c}\mu_{\hat{u}_c} + C_1)(2\sigma_{u_c\hat{u}_c} + C_2)}{(\mu_{u_c}^2 + \mu_{\hat{u}_c}^2 + C_1)(\sigma_{u_c}^2 + \sigma_{\hat{u}_c}^2 + C_2)}, \quad \text{where}$$

$$\mu_{u_c} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} u_c(i, j), \quad \sigma_{u_c\hat{u}_c} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} (u_c(i, j) - \mu_{u_c})(\hat{u}_c(i, j) - \mu_{\hat{u}_c})$$

A significant limitation of SSIM arises when it is applied to signed-valued fields. Unlike its original application to nonnegative image intensities, a perfect sign-reversed field ($\hat{u} = -u$) can achieve nearly the same SSIM score as a perfect reconstruction ($\hat{u} = u$). Consequently, a model trained solely with SSIM may converge to physically incorrect solutions containing polarity inversions while still minimizing the loss. The following derivation demonstrates the mathematical origin of this ambiguity.

565 For the purposes of the following discussion, we consider the common case
 566 where the stabilizing constants satisfy

$$C_1 \ll 2\mu_{u_c}^2, \quad C_2 \ll 2\sigma_{u_c}^2,$$

567 allowing them to be neglected.

568 For a correct reconstruction,

$$u_c^{(+)} = u_c,$$

569 the SSIM score is

$$\text{SSIM}(u_c, u_c^{(+)}) = \frac{(2\mu_{u_c}^2)(2\sigma_{u_c}^2)}{(2\mu_{u_c}^2)(2\sigma_{u_c}^2)} = 1.$$

570 Now consider a sign-reversed reconstruction. The field and its mean reverse
 571 sign,

$$u_c^{(-)} = -u_c, \quad \mu_{u_c^{(-)}} = -\mu_{u_c},$$

572 while the covariance reverses sign and the variance remains unchanged,

$$\sigma_{u_c, u_c^{(-)}} = -\sigma_{u_c, u_c}, \quad \sigma_{u_c^{(-)}}^2 = \sigma_{u_c}^2.$$

573 Substituting these relationships into the SSIM expression gives

$$\text{SSIM}(u_c, u_c^{(-)}) = \frac{(-2\mu_{u_c}^2)(-2\sigma_{u_c}^2)}{(2\mu_{u_c}^2)(2\sigma_{u_c}^2)} = 1.$$

574 The numerator therefore differs from that of a correct reconstruction by two
 575 factors of -1 , and thus, under these conditions, SSIM assigns essentially the
 576 same similarity score to a field and its sign-reversed counterpart as it does to
 577 a perfect reconstruction. This ambiguity constitutes a significant limitation
 578 when SSIM is applied to signed-valued regression problems where the polarity
 579 of the solution is physically meaningful. During training, the network frequently
 580 converged to predictions in which localized regions or the entire image were

the negative of the correct solution while still achieving a favorable SSIM objective. Although the resulting fields exhibited accurate interface locations, boundary geometry, and overall morphology, the polarity inversions rendered the predictions physically incorrect. Consequently, SSIM should not be used as the sole training criterion for signed-valued fields unless it is supplemented by an additional loss that explicitly penalizes sign reversals.

- The normalized mean absolute error (NMAE) loss is defined as

$$\mathcal{L}_{\text{NMAE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^4 \frac{|\hat{u}_c(i, j) - u_c(i, j)|}{|u_c(i, j)| + \varepsilon}$$

Unlike MAE, NMAE weights each prediction error by the inverse magnitude of the target value, causing the optimization to place greater emphasis on regions where the ground-truth solution is small in scale. Consequently, the loss seeks to minimize relative error rather than absolute error, preventing low-magnitude features from being dominated by high-magnitude regions during training.

Experimentally, NMAE successfully reduced the relative error in low-amplitude portions of the solution. However, this improvement came at the expense of increased overall MAE and MSE compared with training using the standard MAE loss. For the present application, the absolute magnitude of the prediction error was determined to be of greater practical importance than the relative error. As a result, the improved accuracy in low-valued regions did not outweigh the degradation in overall numerical accuracy, making NMAE a less suitable optimization criterion for the objectives of this work. However, for other applications, NMAE can be a valid and superior training criterion than standard MAE.

- The normalized mean squared error (NMSE) loss is defined as

$$\mathcal{L}_{\text{NMSE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^4 \frac{(\hat{u}_c(i, j) - u_c(i, j))^2}{(u_c(i, j))^2 + \varepsilon}$$

604 Similar to NMAE, NMSE normalizes the prediction error by the magnitude
605 of the target field, causing the optimization to prioritize relative error rather
606 than absolute error. The quadratic penalty retains the characteristic behavior
607 of MSE by assigning greater weight to larger residuals, while the normalization
608 emphasizes regions where the target magnitude is small.

609 Experimentally, NMSE reduced the relative error in low-amplitude regions
610 compared with standard MSE. However, this improvement was accompanied by
611 increased overall validation MAE and MSE, performing worse than conventional
612 MSE optimization. As with NMAE, the objective of this work was to minimize
613 the absolute prediction error rather than the relative error. Consequently, the
614 increased emphasis placed on low-magnitude regions did not translate into
615 improved overall predictive performance, making NMSE less appropriate for
616 the present application.

617 *S1.3. Algorithms for Input Wavelet Encoding*

618 The band and wavevector conditioning channels used by the FNO are generated
619 by the Gabor wavelet embeddings below, which match the implementations in the
620 accompanying code.

Algorithm 1 1D Gabor Wavelet Embedding from Band Index

- 1: **INPUT:** Band index b ; grid size $S \leftarrow 32$; frequency scale $r \leftarrow 2.0$
- 2: Create linspace vectors $x, y \in [-1, 1]$ with S points
- 3: Construct meshgrid $X, Y \leftarrow \text{meshgrid}(x, y)$
- 4: Compute base frequency:

$$f \leftarrow (1.0 + |b|) \cdot \frac{S}{8}$$

- 5: Compute rotation angle:

$$\theta \leftarrow (b \bmod 8) \cdot \frac{S}{16}$$

- 6: Rotate coordinates:

$$X_\theta \leftarrow X \cos \theta + Y \sin \theta, \quad Y_\theta \leftarrow -X \sin \theta + Y \cos \theta$$

- 7: Set Gaussian envelope widths:

$$\sigma_x \leftarrow \sigma_y \leftarrow \frac{0.3}{r}$$

- 8: Compute Gabor wavelet embedding:

$$\psi_b(x, y) \leftarrow \exp \left(-\frac{X_\theta^2}{2\sigma_x^2} - \frac{Y_\theta^2}{2\sigma_y^2} \right) \cdot \cos(f \cdot X_\theta)$$

- 9: **RETURN:** $\psi_b \in \mathbb{R}^{S \times S}$
-

621 In Algorithm ??, the grid size $S = 32$ matches the geometry resolution used by
622 the FNO. For this resolution the base frequency is $f = (1 + |b|) S/8$, so successive
623 bands increase both carrier frequency and orientation. The modulo in θ repeats
624 orientation every eight bands, while f continues to increase, so embeddings remain
625 distinguishable beyond eight bands. The constants $r = 2.0$, $S/16$, and 0.3 were chosen
626 so that the wavelets occupy most of the spatial domain and remain visually distinct
627 across bands.

Algorithm 2 2D Gabor Wavelet Embedding for Wavevectors

- 1: **INPUT:** Wavevector components k_x, k_y (radians); grid size $S \leftarrow 32$; frequency scale $r \leftarrow 1.0$
- 2: Set constants $f_0 \leftarrow 2.20$, $\alpha \leftarrow 41$, $N_x \leftarrow 13$, $N_y \leftarrow 25$
- 3: Create linspace vectors $x, y \in [-1, 1]$ with S points
- 4: Construct meshgrid $X, Y \leftarrow \text{meshgrid}(x, y)$
- 5: Compute frequencies:

$$f_x \leftarrow (f_0 + k_x) \alpha, \quad f_y \leftarrow (f_0 + k_y) \alpha$$

- 6: Compute rotation angles:

$$\theta_x \leftarrow (k_x \bmod N_x) \cdot \frac{\pi}{N_x}, \quad \theta_y \leftarrow (k_y \bmod N_y) \cdot \frac{\pi}{N_y}$$

- 7: Rotate coordinates:

$$X_\theta \leftarrow X \cos \theta_x + Y \sin \theta_y, \quad Y_\theta \leftarrow -X \sin \theta_x + Y \cos \theta_y$$

- 8: Set Gaussian envelope widths:

$$\sigma_x \leftarrow \sigma_y \leftarrow \frac{0.5}{r}$$

- 9: Compute Gabor wavelet embedding:

$$\psi_{k_x, k_y}(x, y) \leftarrow \exp \left(-\frac{X_\theta^2}{2\sigma_x^2} - \frac{Y_\theta^2}{2\sigma_y^2} \right) \cdot \sin(f_x X_\theta) \cdot \sin(f_y Y_\theta)$$

- 10: **RETURN:** $\psi_{k_x, k_y} \in \mathbb{R}^{S \times S}$
-

628 In Algorithm ??, k_x and k_y enter as continuous radian valued wavevector com-
629 ponents. Distinct modulo periods $N_x = 13$ and $N_y = 25$ reduce aliasing under
630 interchange of the two components. The offset f_0 and scale α map the physical
631 range of k into carrier frequencies that remain well resolved on the $S \times S$ grid, while

632 $\sigma = 0.5/r$ sets the envelope width so that the patterns remain spatially localized yet
633 informative in both domains.

634 *S1.4. Wavelet Decoding*

635 For purposes of checking general applicability, an experiment was run to check
636 if a positive scalar may be embedded in a Gabor wavelet image and recovered after
637 pixelization, storage, or inference. Algorithms ?? and ?? implement this encode–
638 decode pair by spectral peak detection with centroid refinement. In the experiment,
639 we test recovery over $[1, 8000]$, spanning about four orders of magnitude. Despite
640 discretization onto a finite $S \times S$ grid and limited numeric precision associated with
641 float16 storage and arithmetic, the decoded values remain accurate across the full
642 range. Figures ?? and ?? show representative encodings and the corresponding round
643 trip error.

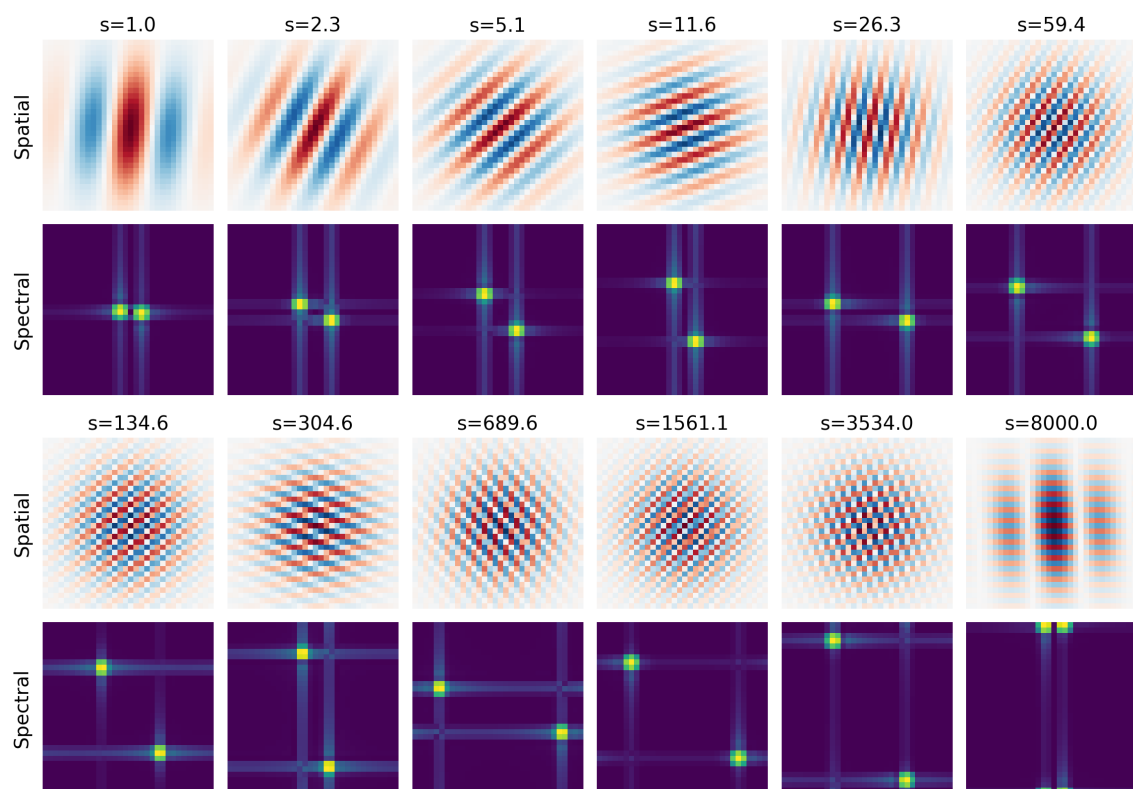


Figure S1: Log spaced Gabor wavelet encodings of scalars from 1 to 8000. Alternating rows show the spatial wavelet images and the corresponding Fourier magnitude spectra.

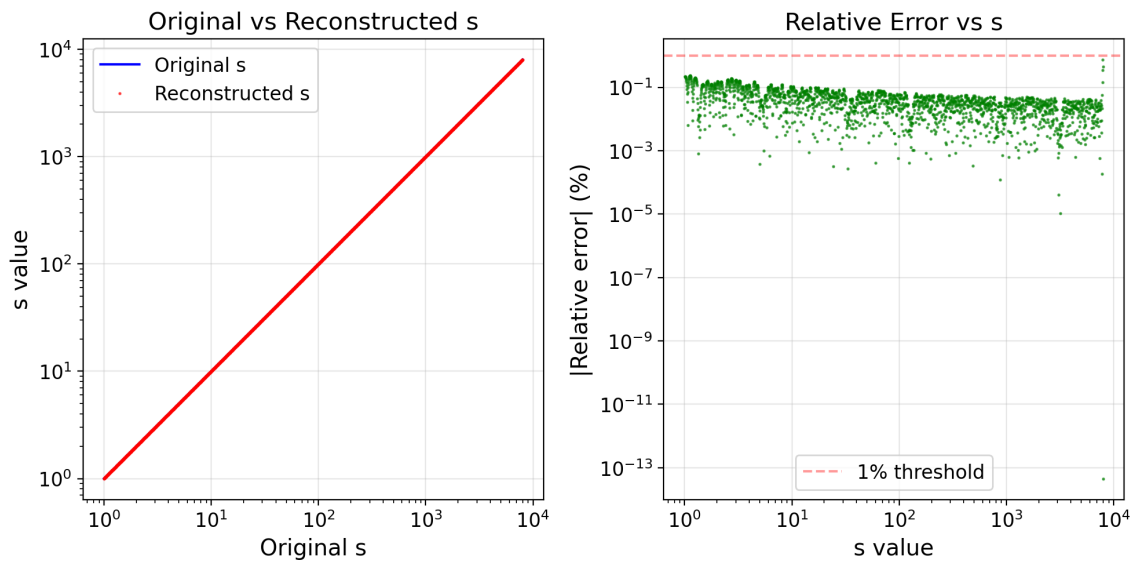


Figure S2: Encode–decode error for scalars from a minimum of 1 to a maximum of 8000.

Algorithm 3 Scalar Wavelet Encoding

- 1: **INPUT:** Scalar $s > 0$; image size $S \leftarrow 32$; bounds $[s_{\min}, s_{\max}]$; spectral-index bounds $[k_{\min}, k_{\max}]$
- 2: Set constants $\gamma \leftarrow 1$, $\phi \leftarrow 0$, $\sigma \leftarrow 8$, $\theta_{\min} \leftarrow \pi/30$, $\theta_{\max} \leftarrow \pi/2 - \pi/30$
- 3: Compute log-domain values:

$$\ell_s \leftarrow \log s, \quad \ell_{\min} \leftarrow \log s_{\min}, \quad \ell_{\max} \leftarrow \log s_{\max}$$

- 4: Compute normalized position and spectral index:

$$t \leftarrow \text{clip}\left(\frac{\ell_s - \ell_{\min}}{\ell_{\max} - \ell_{\min}}, 0, 1\right), \quad k \leftarrow k_{\min} + t(k_{\max} - k_{\min})$$

- 5: Compute log width per unit k :

$$\Delta_\ell \leftarrow \frac{\ell_{\max} - \ell_{\min}}{k_{\max} - k_{\min}}$$

- 6: Compute orientation from within-band log position:

$$\theta \leftarrow \theta_{\min} + \left(\frac{(\ell_s - \ell_{\min}) \bmod \Delta_\ell}{\Delta_\ell}\right) (\theta_{\max} - \theta_{\min})$$

- 7: Build centered grid $X, Y \in \{-S/2, \dots, S/2 - 1\}$, then rotate:

$$X_\theta \leftarrow X \cos \theta + Y \sin \theta, \quad Y_\theta \leftarrow -X \sin \theta + Y \cos \theta$$

- 8: Set envelope widths $\sigma_x \leftarrow \sigma$, $\sigma_y \leftarrow \gamma \sigma_x$, and carrier frequency $\omega \leftarrow 2\pi k/S$
- 9: Compute wavelet image:

$$\psi_s \leftarrow \exp\left(-\frac{1}{2} \left(\frac{X_\theta^2}{\sigma_x^2} + \frac{Y_\theta^2}{\sigma_y^2}\right)\right) \cos(\omega X_\theta + \phi)$$

- 10: **RETURN:** $\psi_s \in \mathbb{R}^{S \times S}$, k , θ
-

Algorithm 4 Scalar Wavelet Decoding

- 1: **INPUT:** Wavelet image $\psi \in \mathbb{R}^{S \times S}$; bounds $[s_{\min}, s_{\max}]$, $[k_{\min}, k_{\max}]$, $[\theta_{\min}, \theta_{\max}]$
- 2: Set constants for centroid refinement: radius $R \leftarrow 3$, power $p \leftarrow 2$
- 3: Mean-center image $\psi_c \leftarrow \psi - \text{mean}(\psi)$, then compute the centered 2D discrete Fourier magnitude:

$$\hat{\psi}(k_x, k_y) \leftarrow \sum_{m=0}^{S-1} \sum_{n=0}^{S-1} \psi_c(m, n) \exp\left(-2\pi i \left(\frac{k_x m}{S} + \frac{k_y n}{S}\right)\right), \quad F(k_x, k_y) \leftarrow |\hat{\psi}(k_x, k_y)|$$

- 4: Keep upper-half-plane spectral support (plus positive-frequency center row) and find dominant peak index (r^*, c^*)
- 5: Define symmetric peak partner about center, then compute weighted local centroid around (r^*, c^*) within radius R , using weights $w = F^p$ and excluding points closer to the symmetric partner
- 6: Convert refined centroid to wavevector coordinates (k_x, k_y) , then:

$$k_{\text{ext}} \leftarrow \sqrt{k_x^2 + k_y^2}, \quad \theta_{\text{ext}} \leftarrow \text{atan2}(k_y, k_x) \bmod \pi$$

- 7: Map extracted k to approximate log value:

$$\ell_{\text{approx}} \leftarrow \ell_{\min} + \text{clip}\left(\frac{k_{\text{ext}} - k_{\min}}{k_{\max} - k_{\min}}, 0, 1\right) (\ell_{\max} - \ell_{\min})$$

- 8: Let $\Delta_\ell \leftarrow (\ell_{\max} - \ell_{\min}) / (k_{\max} - k_{\min})$, compute within-band position from angle:

$$\alpha \leftarrow \text{clip}\left(\frac{\theta_{\text{ext}} - \theta_{\min}}{\theta_{\max} - \theta_{\min}}, 0, 1\right), \quad \delta_\ell \leftarrow \alpha \Delta_\ell$$

- 9: Find nearest consistent log value by testing neighboring bands $b-1, b, b+1$:

$$\ell^* \leftarrow \arg \min_{\ell \in \{\ell_{\min} + j\Delta_\ell + \delta_\ell\}} |\ell - \ell_{\text{approx}}|$$

- 10: Clip ℓ^* to $[\ell_{\min}, \ell_{\max}]$, then decode:

$$s_{\text{ext}} \leftarrow \exp(\ell^*)$$

- 11: **RETURN:** $s_{\text{ext}}, k_{\text{ext}}, \theta_{\text{ext}}$
-

644 Algorithms ?? and ?? summarize the forward and inverse mapping used in this
645 experiment. The forward mapping places the scalar on a log scale and jointly encodes
646 magnitude and orientation into a Gabor-like pattern. The inverse mapping recovers an
647 estimate by spectral peak detection with centroid refinement, followed by a consistency
648 step that combines extracted magnitude and angle to reconstruct the most likely log
649 value.